BAMi WMS 技术架构文档

Technical Architecture Documentation

Version: 1.0
Date: March 2026
Prepared for: APOLLOTECH SOFTWARE CORPORATION (Philippines)
Prepared by: BAMi Technology Team

Table of Contents

1. System Overview
2. Technical Architecture
3. System Modules
4. Code Structure
5. Database Architecture
6. API Design
7. Deployment Guide
8. Development Environment Setup

---

1. System Overview

1.1 Introduction

BAMi WMS (Warehouse Management System) is a comprehensive logistics platform designed for multi-warehouse, multi-owner, multi-platform operations. The system supports both B2B and B2C fulfillment scenarios with full batch tracking and inventory management capabilities.

1.2 Core Capabilities

Based on the 9 core modules:

Module	Description	Key Features
Basic Data	Master data management	Owners, warehouses, SKUs, locations
Inbound	Receiving management	ASN, arrival registration, receiving, putaway
Putaway	Inventory placement	System-directed putaway, RF operations
Inventory	Stock management	Transfers, adjustments, cycle counting, freeze/thaw
Replenishment	Stock replenishment	Auto-triggered, manual, priority-based
Return Supply	RTV (Return to Vendor)	Quality returns, supplier returns
Reassignment	Transfer between warehouses	Inter-warehouse transfers
Sales Outbound	Order fulfillment	Wave management, picking, packing, shipping
Inventory Ledger	Transaction tracking	Location/Warehouse ledger, audit trails

1.3 System Highlights

- **Multi-Tenant**: Supports multiple warehouse owners
- **Batch Management**: Production date, expiry date, batch number tracking
- **Full Traceability**: Complete inventory transaction history
- **RF Support**: Mobile device integration for warehouse operations
- **Interception**: 9-point order cancellation with automatic rollback
- **Multi-Platform**: Integrates with e-commerce platforms (TikTok, Shopee, Lazada)

---

2. Technical Architecture

2.1 Overall Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                        Client Layer                          │
├────────────────┬──────────────────┬──────────────────────────┤
│   Web App      │   RF Device      │   External Systems       │
│   (Vue.js)     │   (Vue.js)       │   (REST API)             │
└────────┬───────┴────────┬─────────┴──────────┬───────────────┘
         │                │                    │
         └────────────────┼────────────────────┘
                          │
┌─────────────────────────▼────────────────────────────────────┐
│                     Gateway Layer                            │
│               (Spring Cloud Gateway)                         │
└──────────────────────────┬───────────────────────────────────┘
                           │
         ┌─────────────────┼─────────────────┐
         │                 │                 │
┌────────▼────────┐ ┌──────▼──────┐ ┌────────▼────────┐
│   WMS Core      │ │  Base Data  │ │ User Center     │
│   Services      │ │   Service   │ │   Service       │
└────────┬────────┘ └──────┬──────┘ └────────┬────────┘
         │                 │                 │
         └─────────────────┼─────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────────┐
│                   Data Layer                                 │
├──────────────────┬──────────────────┬──────────────────────────┤
│   MySQL          │   Redis          │   Message Queue          │
│   (Primary DB)   │   (Cache)        │   (Async Processing)     │
└──────────────────┴──────────────────┴──────────────────────────┘
```

2.2 Frontend Technology Stack

Component	Technology	Version	Purpose
Framework	Vue.js	2.7.15	UI Framework
UI Library	Ant Design Vue	1.7.8	Component Library
Router	Vue Router	3.6.5	SPA Navigation
State Management	Vuex	-	State Management

Component	Technology	Version	Purpose
HTTP Client	Axios	1.6.2	API Communication
Charts	ECharts	5.4.3	Data Visualization
Internationalization	Vue i18n	8.28.2	Multi-language Support
Micro-Frontend	Qiankun	2.10.16	Module Integration
Date Handling	Day.js	1.11.10	Date Manipulation
Map Integration	AMap JSAPI	1.0.1	Location Services

Frontend Projects: - `my-web`: Main web application for PC browsers - `my-wms-rf-app`: Mobile RF application for warehouse devices

2.3 Backend Technology Stack

Component	Technology	Version	Purpose
Framework	Spring Boot	2.0.4.RELEASE	Application Framework
Microservices	Spring Cloud	Finchley.SR1	Distributed System
Language	Java	8	Programming Language
Build Tool	Maven	3.x	Dependency Management
Database	MySQL	5.7+	Primary Database
Cache	Redis	5.x	Distributed Cache
ORM	MyBatis	-	Data Access Layer
API Gateway	Spring Cloud Gateway	-	Request Routing
Code Generator	Lombok	1.18.8	Boilerplate Reduction
Logging	Log4j2	-	Logging Framework

2.4 Backend Module Structure

```
后端 (Backend)
├── base-data-parent/           # Basic Data Management
|   ├── base-data-client/       # API Client
|   └── base-data-server/       # Service Implementation
├── boss-crm-parent/            # CRM Module
├── common-parent/              # Common Utilities
|   ├── common-core/            # Core utilities
|   ├── common-web/             # Web utilities
|   └── common-redis/           # Redis utilities
├── open-external-parent/       # External API Integration
|   ├── open-external-client/   # Client interfaces
|   └── open-external-server/   # Server implementation
├── org-permission-parent/      # Organization & Permission
├── usercenter-parent/          # User Management
|   ├── usercenter-client/
|   ├── usercenter-common/
|   └── usercenter-server/
├── web-gateway/                # API Gateway
└── wms-parent/                 # WMS Core Module
    ├── wms-base/               # Base WMS services
    ├── wms-common/             # WMS utilities
    ├── wms-inbound/            # Inbound Management
    ├── wms-inventory/          # Inventory Management
    ├── wms-job/                # Scheduled Jobs
    ├── wms-outbound/           # Outbound Management
    ├── wms-pub/                # Public services
    └── wms-report/             # Reporting Module
```

---

3. System Modules

3.1 Module Correspondence to Business Functions

Business Module	Code Module	Description
Basic Data Management	`base-data-parent`	Owners, warehouses, SKUs, locations
Inbound Management	`wms-inbound`	ASN, receiving, inspection
Putaway Management	`wms-inventory`	Inventory placement

Business Module	Code Module	Description
Inventory Management	`wms-inventory`	Transfers, adjustments, counting
Replenishment	`wms-inventory`	Picking area replenishment
Return Supply	`wms-outbound`	RTV operations
Reassignment	`wms-outbound`	Inter-warehouse transfers
Sales Outbound	`wms-outbound`	Order fulfillment
Inventory Ledger	`wms-report`	Transaction tracking

3.2 Key Domain Entities

3.2.1 Inbound Domain

- `InboundNotify` - Advance Shipping Notice (ASN)
- `InboundNotifyDetail` - ASN line items
- `ArrivalRegistration` - Vehicle/container arrival
- `ReceivingRecord` - Actual receiving records
- `ReceivingDetail` - Received items with batch info

3.2.2 Inventory Domain

- `InventoryLocation` - Location-level inventory
- `InventoryWarehouse` - Warehouse-level inventory
- `InventoryTransaction` - All inventory movements
- `TransferOrder` - Internal transfer orders
- `AdjustmentOrder` - Inventory adjustments
- `CycleCount` - Physical count records

3.2.3 Outbound Domain

- `OutboundOrder` - Sales orders
- `OutboundWave` - Picking waves
- `PickTask` - Picking tasks

- `PackTask` - Packing tasks
- `Shipment` - Shipping records

3.2.4 Base Data Domain

- `Owner` - Warehouse owners/consignors
- `Warehouse` - Warehouse master data
- `WarehouseZone` - Zone definitions
- `WarehouseLocation` - Location master
- `Sku` - Product master data
- `SkuBatch` - Batch tracking info

---

4. Code Structure

4.1 Standard Package Structure

Each backend module follows this structure:

```
module-server/
├── src/
|   ├── main/
|   |   ├── java/
|   |   |   └── com/wms/
|   |   |       ├── controller/      # REST API Controllers
|   |   |       ├── service/         # Business Logic
|   |   |       |   ├── impl/        # Service Implementations
|   |   |       ├── mapper/          # MyBatis Data Access
|   |   |       ├── entity/          # Domain Entities
|   |   |       ├── vo/              # Value Objects (DTOs)
|   |   |       ├── dto/             # Data Transfer Objects
|   |   |       ├── qry/             # Query Objects
|   |   |       ├── handler/         # Business Handlers
|   |   |       ├── excel/           # Excel Import/Export
|   |   |       ├── event/           # Domain Events
|   |   |       └── config/          # Configuration
|   |   └── resources/
|   |       ├── mappers/             # MyBatis XML Mappings
|   |       ├── application.yml      # Main Config
```

```
|   |       └── application-*.yml    # Environment Configs
|   └── test/                        # Unit Tests
└── pom.xml                          # Maven Configuration
```

4.2 Frontend Structure

```
my-web/
├── public/                          # Static Assets
|   ├── static/
|   |   ├── css/                     # Stylesheets
|   |   ├── images/                  # Images
|   |   └── printTemplate/           # Print Templates
├── src/
|   ├── api/                         # API Client Modules
|   ├── assets/                      # Component Assets
|   ├── components/                  # Reusable Components
|   ├── router/                      # Route Definitions
|   ├── store/                       # Vuex Store
|   ├── views/                       # Page Components
|   |   ├── basicData/               # Basic Data Module
|   |   ├── inbound/                 # Inbound Module
|   |   ├── inventory/               # Inventory Module
|   |   ├── outbound/                # Outbound Module
|   |   └── report/                  # Report Module
|   ├── utils/                       # Utilities
|   ├── locale/                      # i18n Translations
|   |   ├── zh-CN/                   # Chinese
|   |   └── en-US/                   # English
|   └── App.vue                      # Root Component
├── package.json                     # NPM Dependencies
└── vue.config.js                    # Vue CLI Config
```

4.3 Key Configuration Files

Application Configuration (application.yml)

```
server:
  port: 8080

spring:
  application:
```

```
    name: wms-server
  datasource:
    url: jdbc:mysql://localhost:3306/wms_db?useUnicode=true&characterEncoding=utf-8
    username: wms_user
    password: ${DB_PASSWORD}
    driver-class-name: com.mysql.jdbc.Driver
  redis:
    host: localhost
    port: 6379
    password: ${REDIS_PASSWORD}
    database: 0

mybatis:
  mapper-locations: classpath:mappers/*.xml
  type-aliases-package: com.wms.entity

logging:
  config: classpath:log4j2.xml
```

---

5. Database Architecture

5.1 Core Tables by Module

Basic Data Module

- `base_owner` - Owner/Consignor information
- `base_warehouse` - Warehouse master
- `base_warehouse_zone` - Zone definitions
- `base_warehouse_location` - Location master
- `base_sku` - Product master
- `base_sku_unit` - SKU unit conversions
- `base_carrier` - Carrier information
- `base_express` - Express company config

Inbound Module

- `inbound_notify` - ASN header

- `inbound_notify_detail` - ASN lines
- `inbound_arrival` - Arrival registration
- `inbound_receiving` - Receiving records
- `inbound_putaway` - Putaway tasks
- `inbound_quality` - Quality inspection

Inventory Module

- `inv_location_stock` - Location inventory
- `inv_warehouse_stock` - Warehouse inventory
- `inv_transaction` - Transaction log
- `inv_transfer` - Transfer orders
- `inv_adjustment` - Adjustment records
- `inv_cycle_count` - Physical count
- `inv_freeze` - Frozen inventory

Outbound Module

- `outbound_order` - Sales orders
- `outbound_order_detail` - Order lines
- `outbound_wave` - Picking waves
- `outbound_wave_detail` - Wave allocations
- `outbound_pick_task` - Picking tasks
- `outbound_pack_task` - Packing tasks
- `outbound_shipment` - Shipments

5.2 Key Design Patterns

1. **Multi-Tenant**: All tables have `owner_id` and `warehouse_id` columns
2. **Batch Tracking**: Key fields - `production_date`, `expiry_date`, `batch_no`
3. **Soft Delete**: `is_deleted` flag on all tables
4. **Audit Fields**: `create_time`, `create_by`, `update_time`, `update_by`

---

6. API Design

6.1 API Conventions

- **Base Path**: `/api/v1/{module}/{resource}`
- **Authentication**: JWT token in Authorization header
- **Response Format**:

```
{
  "code": 200,
  "message": "success",
  "data": {},
  "success": true
}
```

6.2 Key API Endpoints

Module	Endpoint	Description
Inbound	POST /api/v1/inbound/notify	Create ASN
Inbound	PUT /api/v1/inbound/notify/{id}/confirm	Confirm ASN
Inbound	POST /api/v1/inbound/receiving	Execute receiving
Inventory	GET /api/v1/inventory/location	Query location stock
Inventory	POST /api/v1/inventory/transfer	Create transfer
Outbound	POST /api/v1/outbound/order	Create order
Outbound	POST /api/v1/outbound/wave	Create wave
Outbound	POST /api/v1/outbound/pick	Execute picking
BaseData	GET /api/v1/base/sku	Query SKUs
Report	GET /api/v1/report/ledger	Inventory ledger

---

7. Deployment Guide

7.1 System Requirements

Minimum Requirements

Component	Specification
CPU	4 cores
RAM	8 GB
Disk	100 GB SSD
OS	Linux CentOS 7+ / Ubuntu 18+
Java	JDK 1.8+
MySQL	5.7+
Redis	5.0+
Nginx	1.18+

Recommended Requirements

Component	Specification
CPU	8 cores
RAM	16 GB
Disk	500 GB SSD
Java	JDK 1.8
MySQL	8.0
Redis	6.0+

7.2 Deployment Architecture

```
                    ┌──────────────┐
                    |   Nginx      |  ← Load Balancer
                    |  (80/443)    |
                    └──────┬───────┘
                           |
            ┌──────────────┼──────────────┐
            |              |              |
    ┌───────▼───────┐ ┌────▼──────┐ ┌─────▼─────┐
    |  Web App      | |  Web App  | |  Web App  |
    |  (Node 1)     | |  (Node 2) | |  (Node 3) |
    └───────┬───────┘ └────┬──────┘ └─────┬─────┘
            |              |              |
            └──────────────┼──────────────┘
                           |
                    ┌──────▼──────┐
                    |   Gateway   |
                    |  (Spring)   |
                    └──────┬──────┘
                           |
    ┌──────────────────────┼──────────────────────┐
    |                      |                      |
┌───▼────┐ ┌──────────────▼──────────────┐ ┌──────▼──────┐
|  WMS   | |      Other Services         | |   Redis     |
|Service | |  (BaseData, UserCenter)     | |   Cluster   |
└───┬────┘ └─────────────────────────────┘ └─────────────┘
    |
┌───▼────┐
| MySQL  |
|(Master)|
└────────┘
```

7.3 Step-by-Step Deployment

Step 1: Environment Preparation

```
# Install Java 8
yum install java-1.8.0-openjdk java-1.8.0-openjdk-devel

# Install MySQL 5.7
wget https://dev.mysql.com/get/mysql57-community-release-el7-11.noarch.rpm
rpm -Uvh mysql57-community-release-el7-11.noarch.rpm
yum install mysql-community-server
```

```
systemctl start mysqld

# Install Redis
yum install redis
systemctl start redis

# Install Nginx
yum install nginx
systemctl start nginx
```

Step 2: Database Setup

```
# Create database
mysql -u root -p
CREATE DATABASE wms_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'wms_user'@'%' IDENTIFIED BY 'your_password';
GRANT ALL PRIVILEGES ON wms_db.* TO 'wms_user'@'%';
FLUSH PRIVILEGES;

# Execute schema scripts
mysql -u wms_user -p wms_db < schema.sql
mysql -u wms_user -p wms_db < init_data.sql
```

Step 3: Backend Deployment

```
# Build the project
cd /path/to/wms-parent
mvn clean package -DskipTests

# Deploy each service
nohup java -jar wms-base-server/target/wms-base-server.jar > wms-base.log 2>&1 &
nohup java -jar wms-inbound-server/target/wms-inbound-server.jar > wms-inbound.log 2>&1
nohup java -jar wms-outbound-server/target/wms-outbound-server.jar > wms-outbound.log 2>
nohup java -jar wms-inventory-server/target/wms-inventory-server.jar > wms-inventory.log
```

Step 4: Frontend Deployment

```
# Build frontend
cd /path/to/my-web
npm install
```

```
npm run build:prod

# Deploy to Nginx
cp -r dist/* /usr/share/nginx/html/

# Configure Nginx
vi /etc/nginx/conf.d/wms.conf
```

Nginx Configuration:

```
server {
    listen 80;
    server_name wms.yourdomain.com;

    location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://localhost:8080/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

7.4 Docker Deployment (Alternative)

```
version: '3'
services:
  mysql:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wms_db
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql

  redis:
```

```
    image: redis:6-alpine
    ports:
      - "6379:6379"

  wms-service:
    image: bami/wms:latest
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/wms_db
      SPRING_REDIS_HOST: redis
    ports:
      - "8080:8080"
    depends_on:
      - mysql
      - redis

  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
    volumes:
      - ./dist:/usr/share/nginx/html
      - ./nginx.conf:/etc/nginx/conf.d/default.conf
    depends_on:
      - wms-service

volumes:
  mysql_data:
```

---

8. Development Environment Setup

8.1 Prerequisites

- JDK 1.8+
- Maven 3.6+
- Node.js 14+
- MySQL 5.7+
- Redis 5.0+
- IntelliJ IDEA (Recommended)
- VS Code (for frontend)

8.2 Backend Setup

```
# Clone repository
git clone git@github.com:lee18926200599-dotcom/bami-wms.git
cd bami-wms

# Import into IntelliJ IDEA
# File -> Open -> Select pom.xml

# Build project
mvn clean install

# Run services individually
# Right-click Application.java -> Run
```

8.3 Frontend Setup

```
cd my-web

# Install dependencies
npm install

# Run development server
npm run serve:dev

# Access application
# http://localhost:8081
```

8.4 Configuration for Development

Backend (application-dev.yml):

```
server:
  port: 8080

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/wms_db_dev
    username: root
    password: root
```

```
  redis:
    host: localhost
    port: 6379

logging:
  level:
    com.wms: DEBUG
```

Frontend (.env.development):

```
VUE_APP_BASE_API=http://localhost:8080/api/v1
VUE_APP_ENV=development
```

Appendix A: Glossary

Term	Description
ASN	Advance Shipping Notice - Pre-arrival notification
SKU	Stock Keeping Unit - Product identifier
RF	Radio Frequency - Handheld device
RTV	Return to Vendor
FIFO	First In, First Out
FEFO	First Expired, First Out
WMS	Warehouse Management System
OMS	Order Management System
ERP	Enterprise Resource Planning

Appendix B: Contact Information

BAMi Technology - Website: www.bamitims.com - Project Lead: [Your Name] - Technical Support: [Email]

Document Version: 1.0

Last Updated: March 4, 2026

Classification: Confidential - For APOLLOTECH Internal Use Only